DH2323 Computer Graphics and Interaction

Simple Texture Mapping Project Report

Markus Wingmyr
wingmyr@kth.se
20040625-4011

May 2026

1 Introduction

In computer graphics, texture mapping is a fundamental technique used to enhance the visual complexity of 3D objects without the added computational cost of adding details to the geometry. By wrapping a 2D image, a texture, around 3D geometry it is possible to greatly increase the visual fidelity of the rendered scene without much cost.

Of course, in order to wrap a 2D image around a 3D object you need to know which pixel in the image go where on the 3D geometry. This is where texture mapping becomes necessary. This report describes the implementation of adding texture mapping to an existing rasterizer.

2 Implementation

The foundation of the project is the rasterizer constructed in lab 2 of the rendering track in the course.

2.1 Texture Coordinates

To map a texture onto geometry, we first need a way to index texture data. For this purpose, we define texture coordinates (u, v) for each point on the texture image. Instead of using absolute pixel coordinates, we normalize the values such that $u, v \in [0, 1]$. Normalizing the vector coordinates means we do not have to know the exact pixel coordinates in the texture image which makes the mapping easier to work with, mainly by allowing the code to be independent of the underlying image resolution.

To load images into memory, we use the stb_image library by Sean Barrett [1]. Since the library loads images as flat arrays, we convert (u, v) into denormalized pixel coordinates (u', v') by scaling u by image width and v by the height . We then compute the 1D index of the pixel as:

$$(v' \cdot \text{ImageWidth} + u') \cdot 3$$

The multiplication by 3 accounts for the fact that each pixel is stored as three consecutive values in memory (red, green, and blue).

2.2 Mapping Textures to Vertices

The basis for mapping a texture to a surface is determining which vertex corresponds to what point in the texture. As such, for each vertex of each triangle, we define a set of texture coordinates (u, v). For simplicity we also assume all textures are square. An example mapping can be seen in Fig. 1

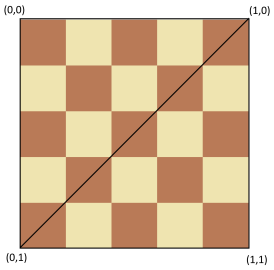

Figure 1: Example of mapping triangle vertices to texture coordinates

To avoid having to manually add a texture mapping for every triangle in every square in the scene we use a helper function AddSquare that takes 4 points and constructs a square from two triangles with a fixed texture mapping and a given texture image.

2.3 Mapping Textures to Surfaces

Now that we have a mapping between the surface's vertices and the texture image, we need to map the rest of the points on the surface to a point in the texture image. For the implementation of this we take inspiration from the methods for 2D texture mapping described by Paul Heckbert [2], specifically we will be interpolating the (u, v) coordinates across each line of each surface.

The original rasterizer already draws surfaces row-by-row so the easiest way to add (u, v) is naturally to extend the Interpolate function to calculate a (u, v) for each pixel. An initial thought might be to simply interpolate (u, v) as such:

$$(u_i, v_i) = (u, v) + (du, dv) * i$$

however, as we can see in Fig. 2 this does not give a great result.

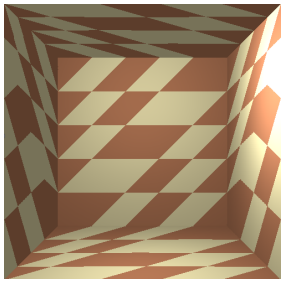

Figure 2: Image output after linearly interpolating (u, v)

The reason for this is that (u, v) are not linear in screen space and as such interpolating them linearly will not give a perspective correct mapping. This is the same issue as linearly interpolating the pixels' 3D position in the original lab assignment. Thus we can extrapolate that since $\frac{\text{pos3d}}{z}$ is linear in screen space $(\frac{u}{z}, \frac{v}{z})$ should also be linear in screen space. Heckbert also suggests interpolating (uq, vq) where uq and vq are linear in x and y, in our case $q = z^{-1}$.

After interpolating (u, v) in the same way we interpolated pos3d in the lab we get the results shown in Fig. 3. Clearly this is much more desirable.

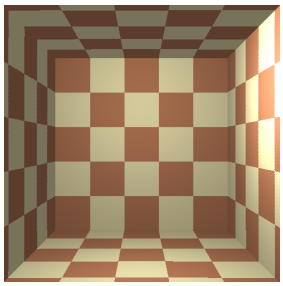

Figure 3: Perspective correct texture mapping

The rasterizer still retains the same functionality as in the lab allowing us to add even more geometry like cubes (Fig. 4) and since textures are defined per triangle we can also give that cube its own texture (Fig. 5).

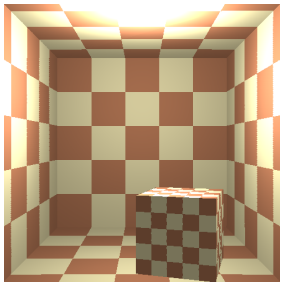

Figure 4: Adding a cube

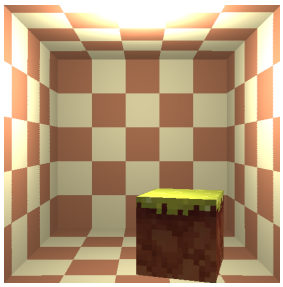

Figure 5: Giving a texture to the cube. The grass block texture is part of the Good Vibes texture pack for Minecraft [3]

2.4 Lighting bug

Comparing Fig. 3 and Fig. 4 one can see that the lighting looks different. This was not intentional, but instead an issue with how the normal for the surfaces was calculated. Specifically, the issue was that the AddSquare helper function I used defined vertices in a different order from the one the normal calculation expected. The

fix for this was simply to manually change the order of the vertices to make the normal calculation correct again.

3 Future Work

Having to set the vertices for a square in a specific order is probably not an ideal setup so a future improvement of the rasterizer could be a way to define a square that can account for any order of vertices.

This rasterizer is quite simple though there are several fundamental aspects of it that could be evaluated through user perception studies (UPS) to inform how to improve both the visual fidelity and computational cost.

While this rasterizer could be seen as mathematically correct it does not take into account that human perception is not fully mathematical. UPS could inform future development of the rasterizer by giving an understanding of how the output is generally perceived by humans. For example maybe the edges look jagged because the rasterizer does not implement anti-aliasing or maybe the light looks wrong, or maybe there are subtle visual artifacts. Of course personally I think it looks perfect but I am a very small sample of individuals.

Though, if users do not perceive these issues there is no inherent need to fix them. In fact, another way UPS can be useful is to find out what a user will not perceive in order to know what computational shortcuts could be taken to make rendering cheaper. For example one could let users observe some piece of geometry moving away from the camera where the texture resolution decreases, at varying levels of aggressiveness, as the object moves away to find a balance between good looking visuals and computational cost.

References

[1] Sean Barrett. *stb*. `https://github.com/nothings/stb`. Accessed: 2026-05-12. 2014.

[2] Paul Heckbert. "Fundamentals of Texture Mapping and Image Warping". In: (May 1998).

[3] Acaitart. *GoodVibes Voxel Asset Pack*. Distributed under CC BY via Phyronnaz/VoxelAssets repository. Accessed: 2026-05-12. 2021. URL: `https://github.com/Phyronnaz/VoxelAssets/tree/master/GoodVibes`.